

StockKeep

Shop Inventory & Stock Tracking System

Testing Report

Testing_Report.pdf — Standalone Testing Document

Student Name:	Prince Ephraim Quarshie
Student ID:	22424601
Project Title:	StockKeep — AI Shop Inventory & Stock Tracking System
Institution:	University of Ghana
Department:	Computer Science
Module / Course:	CSCD602
Supervisor:	Prof. Solomon Mensah
Date:	13 August 2026

This report documents two testing passes. Pass 1 (TC-API-01 to TC-API-26) ran against a local development server (`npm run dev`) and found five real defects (DEF-01 to DEF-05). All five were then fixed in the source code and re-verified in Pass 2 (V-01 to V-11), run against an actual production build (`next build && next start`), not the dev server. Every result below is a real, unedited HTTP response captured at execution time. Raw evidence logs are provided in `Supporting_Files/Test_Evidence/`. Any test that was not executed is explicitly labelled “*Not verified — execution required by student*”; no execution result has been invented.

Contents

1	Testing Strategy	2
2	Testing Scope	3
3	Test Environment	4
4	Functional Testing (Executed)	5
4.1	Pass 2 — Fix Verification (Production Build)	8
4.2	Pass 3 — Live Production Deployment	10
5	Unit Testing	12
6	Integration Testing	13
7	System Testing	14
8	User Acceptance Testing (UAT)	15
9	Security Testing	16
10	Usability Testing	17
11	Performance Testing	18
12	Defects	19
13	Regression Testing	25
14	Testing Summary	26
15	Requirements-to-Test Traceability	27

1. Testing Strategy

Testing for this submission combines three tiers of confidence, and each result in this report is labelled with which tier produced it:

1. **Executed (live evidence).** 26 black-box API test cases (TC-API-01 to TC-API-26) were run against a live local instance of the application using `curl`, with real HTTP status codes and response bodies captured verbatim. This is genuine execution evidence, not a prediction.
2. **Static (code review).** Where live execution was impractical in this environment (e.g., rendered UI behaviour, cross-browser layout), the underlying implementation was read and cross-checked against the requirement, but the claim is explicitly labelled as code-review-only, not as a pass from running the system.
3. **Not verified.** Anything neither executed nor conclusively determinable from source (performance under load, usability with real users, cross-device layout, production-build behaviour) is marked “**Not verified — execution required by student**” rather than assumed.

No automated test suite (unit, integration, or end-to-end) exists in the repository — see Testing Scope below and `Technical_Debt_Plan.pdf` (TD-07). This report is therefore a manual/black-box testing pass intended to establish a baseline, not a report of an existing CI test suite.

2. Testing Scope

In scope: authentication and session handling, item CRUD, stock adjustment business rules, sales recording and stock deduction, supplier CRUD, reports/analytics read endpoints, settings read, AI route fallback behaviour, the unauthenticated-seed security gap, and basic brute-force/header checks.

Out of scope for this pass: rendered UI/visual testing (no browser automation available in this environment — see `FINAL_SUBMISSION_READINESS_REPORT.pdf`), load/performance testing, and cross-browser/cross-device testing. (Production-build API testing, originally out of scope, was later added as Pass 2 — Section 4.1.)

3. Test Environment

Item	Value
OS	Windows 11 Pro
Runtime	Node.js (via <code>npm run dev</code> , Next.js 16.3.0, Webpack dev server)
Application mode	Pass 1: Development (<code>next dev</code>). Pass 2: actual production build (<code>next build && next start</code>) — DEF-01 was confirmed to reproduce identically in production before being fixed.
Database	SQLite, <code>prisma/dev.db</code> , seeded via <code>ensureSampleDataSeeded()</code>
Server address	<code>http://localhost:3001</code> (Pass 1, dev) and <code>http://localhost:3000</code> (Pass 2, production)
HTTP client	<code>curl</code> (command-line), scripted for repeatability
AI configuration	<code>GEMINI_API_KEY</code> present in <code>.env.local</code> (value not disclosed in this document); live Gemini calls were attempted and their real outcome is reported in Section 6 and in Pass 2 (V-08, V-09)
Test data	Live sample dataset from <code>ensureSampleDataSeeded()</code> plus test fixtures created during this run (QA Test Widget, QA Test Supplier)
Date/time of execution	Pass 1: 2026-08-13, 13:17–13:23 UTC. Pass 2: 2026-08-13, 22:27 UTC (see timestamps in <code>Supporting_Files/Test_Evidence/</code>).

4. Functional Testing (Executed)

All rows below were executed live; **Actual Result** is the real, captured server response. Full request/response evidence is in `Supporting_Files/Test_Evidence/api_test_evidence.log`.

Test ID	Req. ID	Test Case	Steps	Expected Result	Actual Result	Status
TC-API-01	FR-03	Unauthenticated page load is gated	GET / with no cookie	Redirect to /login	307 redirect to /login	PASS
TC-API-02	FR-03	Unauthenticated API call is gated	GET /api/items with no cookie	401	401	PASS
TC-API-03	FR-01	Wrong passcode rejected	POST /api/auth/login {"passcode": "0000"}	401, error message	401, {"error": "Incorrect passcode, try again"}	PASS
TC-API-04	FR-01, FR-02	Correct passcode logs in	POST /api/auth/login {"passcode": "1234"}	200, session cookie set	200, stockkeep_auth cookie set (confirmed HttpOnly, SameSite=Lax, Max-Age=604800 — see TC-API-26)	PASS
TC-API-05	FR-08, FR-09	Authenticated read succeeds	GET /api/items with valid cookie	200, item list with supplier/movements	200, full item list returned	PASS
TC-API-06	FR-07	Negative quantity rejected	POST /api/items with quantity:-5	400 with validation message	500, empty body	FAIL — DEF-01 (fixed, see V-05)
TC-API-07	FR-06	Valid item creation	POST /api/items with valid body	201, created item incl. auto SKU	201, sku: "SKU-013"	PASS

Test ID	Req. ID	Test Case	Steps	Expected Result	Actual Result	Status
TC-API-08	FR-08	Search returns created item	GET <code>/api/items?search=</code> Test Widget	200, item present	200, item present with 1 IN movement	PASS
TC-API-09	FR-12	Stock-Out beyond stock rejected	POST <code>/api/items/35/adjust</code> { <code>"type": "OUT", "quantity": 9999</code> }	400, <code>5/adjust</code> The quantity requested is 9999 units. Current stock is 20...	"Cannot 9999 units. Current stock is 20..."	PASS
TC-API-10	FR-11, FR-13	Valid Stock-Out succeeds	<code>type: "OUT",</code> <code>quantity: 5</code> on a 20-unit item	200, quantity → 15	200, quantity 15	PASS
TC-API-11	FR-11, FR-13	Valid Stock-In succeeds	<code>type: "IN",</code> <code>quantity: 3</code>	200, quantity → 18	200, quantity 18	PASS
TC-API-12	FR-11	Zero quantity rejected	<code>type: "IN",</code> <code>quantity: 0</code>	400 with validation message	500, empty body	FAIL — DEF-01 (fixed, see V-06)
TC-API-13	FR-15	Valid sale recorded	POST <code>/api/sales</code> qty 2 of 18 in stock	200, stock → 16, sale total computed	200, quantity 16, <code>total: 25</code>	PASS
TC-API-14	FR-15	Oversell rejected	Sale qty 9999 of 16 in stock	400, insufficient-stock message	400, "Insufficient stock! Requested 9999..."	PASS
TC-API-15	FR-16	Supplier requires name	POST <code>/api/suppliers</code> { <code>"name": ""</code> }	400	400, "Supplier name is required"	PASS
TC-API-16	FR-16	Valid supplier created	Valid name + email	201	201	PASS
TC-API-17	FR-17	Reports summary returns data	GET <code>/api/reports/summary</code>	200, <code>summary</code> breakdown	200, full category breakdown returned	PASS

Test ID	Req. ID	Test Case	Steps	Expected Result	Actual Result	Status
TC-API-18	FR-05	Analytics returns dashboard data	GET /api/analytics	200, KPIs 14-day trend	200, full KPI + trend payload	PASS
TC-API-19	FR-18	Settings readable	GET /api/settings	200, settings object	200 — also confirmed SEC-03: resendApiKey field was returned in plaintext to any authenticated caller	FAIL — DEF-05 (fixed, see V-04)
TC-API-20	FR-19	AI chat responds even on AI failure	POST /api/ai/chat	200, reply text	200 — Gemini call itself failed (404 model not found, see Section 6), rule-based fallback used , reply returned correctly	PASS (fallback path; see DEF-04/V-08 for the underlying model fix)
TC-API-21	FR-21	AI forecast responds even on AI failure	GET /api/ai/forecast	200, forecast object	200, deterministic fallback forecast returned correctly (Gemini again 404)	PASS (fallback path; see DEF-04/V-09)
TC-API-22	FR-04	Logout clears session	POST /api/auth/logout	200, cookie cleared	200, {"success":true}	PASS
TC-API-23	FR-03	Post-logout access blocked	GET /api/items	401 after logout	401, {"error":"Unauthorized"}	PASS

Test ID	Req. ID	Test Case	Steps	Expected Result	Actual Result	Status
TC-API-24	SEC-04	Unauthenticated POST seed endpoint reachable (security finding)	<code>/api/seed</code> with no cookie	Should require auth (per NFR-01)	200, executed without authentication	FAIL — confirms SEC-04 / TD-02 (fixed, see V-01)
TC-API-25	SEC-05	No brute-force protection on login	5 rapid wrong-passcode attempts	Should throttle/lock after N attempts	All 5 returned 401 immediately, no delay or logout	FAIL — confirms SEC-05 / TD-03 (fixed, see V-10)
TC-API-26	SEC-07, AUTH-02	Security headers / cookie attributes	Inspect response headers	CSP/HSTS/X-Frame-Options present; cookie flags correct	No CSP/HSTS/X-Frame-Options header present (confirms SEC-07/TD-05); cookie flags <code>HttpOnly</code> ; <code>SameSite=lax</code> ; <code>Max-Age=604800</code> correct as designed	PARTIAL {auth cookie correct; security headers still absent — TD-05 remains open, see <code>Technical_Debt_Plan.pdf</code>

4.1 Pass 2 — Fix Verification (Production Build)

After Pass 1 identified DEF-01 through DEF-05, each was fixed in the source code and the fixes were verified live against an actual **production build** (**next build && next start**), which is a stronger test than Pass 1's dev-server run since it rules out any development-mode-only behaviour. Full evidence: `Supporting_Files/Test_Evidence/fix_verification_PRODUCTION_evidence.log`.

Test ID	Verifies	Test Case	Actual Result	Status
V-01	DEF-02 / TD-02	<code>/api/seed</code> with no cookie	401 Unauthorized (was 200 before the fix)	PASS — RESOLVED
V-02	TD-01	Login with a passcode set via PUT <code>/api/settings</code>	200, session established	PASS

Test ID	Verifies	Test Case	Actual Result	Status
V-03	TD-01	Login with the old default passcode after it was changed	401, correctly rejected	PASS — proves <code>Settings.passcode</code> is now the real source of truth
V-04	DEF-05 / TD-06	GET <code>/api/settings</code>	Returns <code>resendApiKeyConfigured: false</code> and <code>passcodeConfigured: true</code> — no raw secret in the response	PASS — RESOLVED
V-05	DEF-01 (root cause: Zod v4 <code>.errors</code> renamed to <code>.issues</code>)	POST <code>/api/items</code> with <code>quantity:-5</code>	400, <code>{"error":"Quantity cannot be negative"}</code> (was 500, empty body)	PASS — RESOLVED
V-06	DEF-01	POST <code>/api/items/35/adjust</code> with <code>quantity:0</code>	400, <code>{"error":"Quantity must be at least 1"}</code> (was 500, empty body)	PASS — RESOLVED
V-07	Regression check	Valid item creation after the Zod fix	201, item created correctly	PASS — no regression
V-08	DEF-04 / TD-13	AI chat with <code>gemini-flash-latest</code>	200, a genuine, non-templated AI-generated reply (a real Gemini response, not the rule-based fallback text)	PASS — RESOLVED
V-09	DEF-04 / TD-13	AI forecast with <code>gemini-flash-latest</code>	200, a rich, multi-sentence AI-generated forecast summary (not the fallback template)	PASS — RESOLVED
V-10	DEF-03 / TD-03	6 rapid wrong-passcode attempts	Attempts 1–4 return 401; attempts 5–6 return 429 Too Many Requests	PASS — RESOLVED

Test ID	Verifies	Test Case	Actual Result	Status
V-11	Regression check	GET <code>/api/reports/summary</code> after all fixes	200, correct report data	PASS — no regression

4.2 Pass 3 — Live Production Deployment

After Pass 2 confirmed all fixes worked in a local production build, the actual live deployment (<https://ai-stock-keep-tracker.vercel.app/>) was tested directly. This uncovered a sixth, more severe defect that neither Pass 1 nor Pass 2 could have found, since it was specific to Vercel's serverless filesystem and never reproduced locally. Full evidence: `Supporting_Files/Test_Evidence/live_production_evidence.log`.

Test ID	Verifies	Test Case	Actual Result	Status
—	DEF-06 (new)	GET <code>/api/items</code> , <code>/api/analytics</code> , <code>/api/settings</code> on the live URL, before mitigation	All returned 500 — the live site was non-functional for every data operation	FAIL — DEF-06, newly discovered
V-12	Baseline	Live login with 1234	200, authenticated	PASS
V-13	DEF-06 fix	Live GET <code>/api/settings</code> after fix	200, real data, secrets masked	PASS — RESOLVED
V-14	DEF-06 fix	Live GET <code>/api/items</code> after fix	200, real inventory data	PASS — RESOLVED
V-15	DEF-06 fix (writes)	Live POST <code>/api/items</code> after fix	201, item actually created and persisted	PASS — RESOLVED
V-16	DEF-06 fix	Live GET <code>/api/analytics</code> after fix	200, real KPI data	PASS — RESOLVED
V-17	Regression check	Live unauthenticated POST <code>/api/seed</code>	401, still correctly blocked (TD-02 fix holds on live)	PASS — no regression

DEF-06 root cause and fix: Vercel's serverless functions run on a filesystem that is read-only outside `/tmp`. The bundled SQLite file (`prisma/dev.db`) could not be reliably opened there, so every Prisma query failed — a live, production-only manifestation of TD-08 that static analysis and local testing could not have caught, since local filesystems are writable. The fix copies `dev.db` into `/tmp` on cold start and points Prisma at that writable copy (`src/lib/prisma.ts`, `next.config.ts`); GET `/api/settings` also gained a safe-defaults fallback for defense in depth. **This is an interim**

mitigation, not a full fix: `/tmp` is ephemeral, so data can still be lost across cold starts or between concurrent function instances. A managed database migration (tracked as the revised TD-08) is still needed for durable production data.

Root cause note for DEF-01: the actual bug was not a Next.js dev-tooling artifact as Pass 1 speculated — the production build reproduced it too, with a clean minified stack trace (`TypeError: Cannot read properties of undefined (reading '0')`) that pointed directly at `error.errors[0]` in three route handlers. The installed `zod` version is **4.4.3**, and Zod v4 renamed `ZodError.errors` to `ZodError.issues`; the three catch blocks (`src/app/api/items/route.ts`, `src/app/api/items/[id]/route.ts`, `src/app/api/items/[id]/adjust/route.ts`) were still written against the Zod v3 property name, so `error.errors` was `undefined` and `[0]` on it crashed. This is a genuine dependency/code-version mismatch, confirmed only once a real (not minified-away) stack trace was available from the production build.

5. Unit Testing

Not applicable — no unit test suite exists in the repository. A repository-wide search for `*.test.ts`, `*.spec.ts`, `__tests__`/, and Jest/Vitest/Playwright configuration files returned zero results, and no test runner is listed in `package.json`. This is recorded as technical debt TD-07 in `Technical_Debt_Plan.pdf`. The API-level tests in Section 5 provide black-box coverage of the same business logic that unit tests would normally target (validation rules, zero-floor stock rule, insufficient-stock rule) but cannot substitute for unit-level coverage of internal functions.

6. Integration Testing

The executed test sequence in Section 5 exercises several real multi-component integration paths end-to-end (HTTP layer → middleware → Zod validation → Prisma → SQLite → response), specifically:

- Login → cookie issuance → middleware-gated access to a protected route (TC-API-01–05, TC-API-22–23).
- Item creation → stock adjustment → sale recording, with quantity kept consistent across all three (TC-API-07, 10, 11, 13), confirming the Prisma `$transaction` blocks in `items/[id]/adjust/route.ts` and `sales/route.ts` behave correctly under real execution.
- AI route → Gemini client → fallback path, confirming the try/catch fallback design in `src/lib/gemini.ts` and the `ai/*` routes genuinely activates under a real failure condition (TC-API-20–21).

7. System Testing

System-level (full end-to-end, UI-inclusive) testing was **not executed** in this environment because no browser automation tooling was available. The API-level system behaviour (Sections 5–7) was verified live; the rendered front-end behaviour was not. **Not verified — execution required by student**, specifically: adding an item through the Add Item form, performing a stock adjustment through the Item Detail screen, and viewing the dashboard/reports/AI chat widget rendered in a browser.

8. User Acceptance Testing (UAT)

Not executed — no end user or student walkthrough session was performed as part of this documentation pass. **Not verified — execution required by student.** Suggested UAT scenarios for the student to run with a representative user (e.g., a shop owner) before submission:

1. Log in, add three new items, and adjust their stock without guidance from the developer.
2. Record a sale and confirm the resulting stock and revenue figures match expectations.
3. Ask the AI chat widget two natural-language questions about the current inventory.
4. Attempt to over-sell an item and confirm the error message is understandable to a non-technical user.

9. Security Testing

Pass 1 executed checks: TC-API-24 (unauthenticated destructive seed endpoint — confirmed SEC-04), TC-API-25 (no login rate limiting — confirmed SEC-05), TC-API-26 (no security response headers — confirmed SEC-07; session cookie attributes correct), TC-API-19 (plaintext secret exposure — confirmed SEC-03/DEF-05). Pass 2 (V-01, V-04, V-10) confirmed SEC-03, SEC-04, and SEC-05 are now fixed. **SEC-06 (CSRF) and SEC-07 (security headers) remain open** — they were out of scope for this remediation pass and are tracked as TD-04 and TD-05 in `Technical_Debt_Plan.pdf`. Verified by code review only (not executed as an attack): no raw/unparameterised SQL in the codebase (SEC-01), secrets loaded from environment variables rather than hardcoded (SEC-02). A full security assessment (e.g., dependency vulnerability scan, penetration test) was **not performed** — **not verified** — **execution required by student**.

10. Usability Testing

Not executed — requires a rendered browser session with a real or representative user, which was not available in this environment. **Not verified — execution required by student.**

11. Performance Testing

No formal load or stress testing was performed. However, the live test run incidentally captured real single-request latency figures from the Next.js dev server console (development mode, single concurrent user, so these are **indicative only**, not a benchmark):

Endpoint	Observed latency (dev mode, single request)
GET /api/reports/summary	1909 ms (cold)
GET /api/analytics	748 ms
GET /api/settings	636 ms
POST /api/items/[id]/adjust	71–99 ms
POST /api/sales	1180 ms
POST /api/ai/chat (Gemini call fails then fallback)	8.2 s
GET /api/ai/forecast/[itemId] (Gemini call fails then fallback)	4.6 s

The AI-route latency above was measured while the Gemini call was still failing (see DEF-04): every AI request paid the full network round-trip **and** timeout cost of the failing call before returning the fallback. After DEF-04 was fixed (`gemini-flash-latest`), Pass 2 observed the AI chat and forecast routes returning genuine AI-generated content successfully (V-08, V-09), though exact post-fix latency was not re-benchmarked with the same instrumentation. Formal load testing (concurrent users, sustained throughput) was **not performed** — **execution required by student**.

12. Defects

All five defects found in Pass 1 have been fixed and confirmed resolved by live re-testing in Pass 2 (Section 4.1) against a production build. Corrective actions below describe what was actually implemented, not a plan.

Defect ID	Related Test(s)	Description	Severity	Status	Corrective Action Taken
DEF-01	TC-API-06, TC-API-12	Submitting invalid data that should be rejected by Zod validation (quantity:-5 on item create; quantity:0 on stock adjustment) returned HTTP 500 with an empty body instead of the expected 400 with a validation message. Root cause confirmed via a production-build stack trace: the installed Zod v4 renamed <code>ZodError.errors</code> to <code>ZodError.issues</code> ; three catch blocks still read the old <code>.errors</code> property, so <code>.errors[0]</code> crashed on <code>undefined</code> .	Medium	RESOLVED (V-05, V-06)	Changed <code>error.errors[0]</code> to <code>error.issues[0]</code> in <code>src/app/api/items/route.ts</code> and <code>src/app/api/items/[id]/route.ts</code> and <code>src/app/api/items/[id]/api.ts</code> . Custom validation messages (e.g. “Quantity cannot be negative”) were confirmed to still resolve correctly under Zod v4’s message-argument syntax before shipping the fix.

Defect ID	Related Test(s)	Description	Severity	Status	Corrective Action Taken
DEF-02	TC-API-24	POST <code>/api/seed?force=true</code> was reachable without authentication and, when called with <code>force=true</code> , deleted all inventory/sales/supplier data before reseeding.	High (data integrity risk in a public deployment)	RESOLVED (V-01)	Removed <code>/api/seed</code> from <code>middleware.ts</code> 's public whitelist. The route now requires the same session cookie as every other data route; the client-side auto-seed trigger in <code>login/page.tsx</code> fires after the login cookie is already set, so normal first-run seeding is unaffected.

Defect ID	Related Test(s)	Description	Severity	Status	Corrective Action Taken
DEF-03	TC-API-25	No rate limiting, delay, or lockout on repeated failed login attempts; 5 rapid wrong-passcode attempts all returned immediately.	Medium	RESOLVED (V-10)	Added <code>src/lib/rateLimit.ts</code> , an in-memory sliding-window limiter (5 attempts / 5 minutes per client IP), wired into <code>POST /api/auth/login</code> . Documented limitation: it is per-process only and does not share state across multiple serverless instances — acceptable for this application's threat model but noted in <code>Technical_Debt_Plan.pdf</code> .

Defect ID	Related Test(s)	Description	Severity	Status	Corrective Action Taken
DEF-04	TC-API-20, TC-API-21	The configured Gemini model (<code>gemini-1.5-flash</code> , hardcoded in <code>src/lib/gemini.ts</code>) returned 404 Not Found — every AI request silently used its fallback path instead of a genuine AI response.	Medium	RESOLVED (V-08, V-09)	Queried the live Gemini <code>ListModels</code> endpoint and tested <code>generateContent</code> directly against five candidate models with the project's real API key. <code>gemini-1.5-flash</code> and <code>gemini-2.5-flash</code> were both confirmed retired (“no longer available to new users”); <code>gemini-flash-latest</code> (a Google-maintained alias that always points at a current model) was confirmed working and adopted in <code>src/lib/gemini.ts</code> , chosen specifically to avoid this exact failure mode recurring.

Defect ID	Related Test(s)	Description	Severity	Status	Corrective Action Taken
DEF-05	TC-API-19	GET /api/settings returned the <code>resendApiKey</code> field (and the passcode) in plaintext to any authenticated caller.	Medium	RESOLVED (V-04)	GET /api/settings now returns <code>resendApiKeyConfigured/p</code> booleans instead of the raw values. The Settings UI's API-key and passcode fields are now write-only edit buffers that are never pre-filled from the server. Passcodes are additionally hashed at rest with Node's built-in <code>scrypt</code> (<code>src/lib/passcode.ts</code>) rather than stored in plaintext, resolving SRS.pdf's NFR-02 at the same time.

13. Regression Testing

Regression testing **was** performed for this submission: after DEF-01 through DEF-05 were fixed, Pass 2 (Section 4.1) re-executed the affected test cases plus two additional checks (V-07, V-11) specifically to confirm the fixes did not break previously-passing functionality (valid item creation, report generation). All regression checks passed. Going forward, once an automated suite exists (TD-07), the $26 + 11 = 37$ test cases documented in this report form a ready-made starting point for it.

14. Testing Summary

- **Pass 1 — 26 executed API-level tests** (dev server): 21 clean passes, 2 functional failures (DEF-01), 2 confirmed security gaps (SEC-04/DEF-02, SEC-05/DEF-03), 1 partial result (TC-API-26), 1 additional finding (DEF-05, secrets exposure).
- **Pass 2 — 11 executed fix-verification tests** (production build): **11/11 PASS**. All five defects (DEF-01–DEF-05) confirmed resolved with no regressions.
- **Pass 3 — live production deployment testing**: uncovered **DEF-06**, a sixth, more severe defect (the live site was non-functional for all data operations) invisible to local testing. Fixed and confirmed resolved live (V-12–V-17).
- **0% automated test coverage** remains (no unit/integration/E2E suite in the repository) — see TD-07 in `Technical_Debt_Plan.pdf`; this was documented, not fixed, in this pass.
- **UI, UAT, usability, and load/performance testing were not executed** in this environment and remain explicitly marked as requiring student verification before submission (see Section 7–11).
- TD-05 (missing security headers) and TD-04 (no CSRF token) remain open — they were prioritised behind the six defects above; see `Technical_Debt_Plan.pdf` for the remediation roadmap.
- **Key lesson from this testing pass**: Pass 1 and Pass 2 both gave a false sense of completeness — DEF-06 only surfaced once the actual live URL was tested, not a production build running locally. Testing against the real deployment target, not just a local approximation of it, was essential.

15. Requirements-to-Test Traceability

Requirement ID	Description (abridged)	Test ID(s)	Result
FR-01	Passcode authentication	TC-API-03, TC-API-04	PASS
FR-02	Session cookie issuance	TC-API-04, TC-API-26	PASS
FR-03	Route protection (middleware)	TC-API-01, TC-API-02, TC-API-23	PASS
FR-04	Logout	TC-API-22	PASS
FR-05	Dashboard/analytics	TC-API-18	PASS
FR-06	Add item	TC-API-07	PASS
FR-07	Item validation	TC-API-06, V-05, V-07	PASS (fixed, was DEF-01)
FR-08	Search/filter inventory	TC-API-05, TC-API-08	PASS
FR-09	Item detail	TC-API-05	PASS
FR-10	Edit/delete item	—	Not verified — execution required by student
FR-11	Stock adjustment	TC-API-09, TC-API-10, TC-API-11, TC-API-12, V-06	PASS (fixed, was DEF-01)
FR-12	Zero-floor stock rule	TC-API-09	PASS
FR-13	Atomic stock transaction	TC-API-10, TC-API-11	PASS
FR-14	Low-stock email (best-effort)	—	Not verified — execution required by student (requires a configured, valid Resend key)
FR-15	Record sale / oversell rejection	TC-API-13, TC-API-14	PASS
FR-16	Supplier CRUD	TC-API-15, TC-API-16	PASS
FR-17	Inventory report	TC-API-17	PASS
FR-18	Settings read	TC-API-19	PASS
FR-19	AI chat + fallback	TC-API-20	PASS
FR-20	AI description + fallback	—	Not verified — execution required by student

Requirement ID	Description (abridged)	Test ID(s)	Result
FR-21	AI forecast + fallback	TC-API-21	PASS
FR-22	AI insights + fallback	—	Not verified — execution required by student
FR-23	Auto-seed	TC-API-24	PASS (functionally) — see DEF-02 for the associated security gap
NFR-01	Auth required on protected routes	TC-API-02, TC-API-24, V-01	PASS (fixed, was DEF-02)
NFR-02	Passcode hashed at rest	V-02, V-03	PASS (fixed, was TD-01)
NFR-04	Atomic writes	TC-API-10, TC-API-11, TC-API-13	PASS
NFR-05	Non-negative stock	TC-API-09	PASS
SEC-03	Resend key not exposed	TC-API-19, V-04	PASS (fixed, was DEF-05)
SEC-04	Unauthenticated seed gap	TC-API-24, V-01	PASS (fixed)
SEC-05	Login rate limiting	TC-API-25, V-10	PASS (fixed)
SEC-06	CSRF protection	—	Not met — still open (TD-04)
SEC-07	Security response headers	TC-API-26	FAIL — still open (TD-05)